

A Dynamic Virtual Memory Management System for LLMs on AI Chips

Gaolin Wei¹, Zhaorui Zhang^{1*}, Jiaqi Xu¹, Chen Jason Zhang¹, Xin Yao², Benben Liu²

¹*Hong Kong Polytechnic University, Hong Kong*

²*The University of Hong Kong, Hong Kong*

Abstract—With the widespread application of large-scale DNN models, efficient training and inference with limited resources has become a popular research area. However, memory fragmentation is a significant barrier to efficient training and inference. In this article, we propose an efficient memory management strategy for execution on the Ascend platform based on the latest NPU virtual memory management APIs and develop efficient memory management logic for these. Indeed, the modification is mainly within the caching allocator of the NPU PyTorch extension, designing related garbage collection, allocation, splitting, re-allocation, and fusing small fragmented blocks. In a four-card study on the Ascend 910B platform with training and inference accelerating architecture, MindSpeed, the greatest performance improvement was achieved by reducing the average fragmentation rate from 10% to 6%.

Index Terms—LLM Training and Fine-Tuning, Virtual Memory Management System, AI Chip.

I. INTRODUCTION

The growing popularity of large language models has necessitated enhanced efficiency in both training and inference [1]–[7]. Training a model from scratch, such as GPT-3, is complex and resource-intensive, requiring approximately 355 GPU years and incurring costs of nearly 4.5 million dollars [8]. Besides the NVIDIA devices, emerging AI chips, such as Ascend, have emerged as a viable alternative to the NVIDIA platform, gaining traction as a second option. However, similar to the challenges faced with NVIDIA, memory fragmentation remains a significant issue on Ascend NPUs.

Although Ascend developed its own machine learning architecture, MindSpeed, PyTorch remains one of the most popular tools among programmers. To accommodate this, Ascend has introduced an extension for PyTorch that supports nearly all the functionalities of native PyTorch. However, the memory management of the native caching allocator is not efficient.

To address the above issues, our approach involves modifying the native PyTorch allocator at a lower level. Ascend provides an interface compatible with PyTorch functions, allowing us to alter the runtime memory module of the native caching allocator, which Ascend subsequently registers. In CANN version 8.0.0, Ascend NPUs support virtual memory management. The native PyTorch extension for Ascend allocates memory using `acrtMalloc` and deallocates it with `acrtFree` [11]. However, these memory-related functions lack flexibility, requiring developers to predetermine the allocation size of

memory segments. During processing, the NPU stream divides the segment into several blocks for data storage. However, the segment space is not always fully utilized, leading to memory fragmentation. Our design involves rewriting the segment and block management logic by utilizing NPU virtual memory management APIs, drawing inspiration from contemporary advanced CUDA optimization techniques. This approach consolidates temporarily inactive sections to fulfill subsequent allocation requests, thereby reducing fragmentation within segments and maximizing memory utilization. Additionally, we have enhanced the basic best-fit algorithm of the PyTorch memory allocator, which minimizes the frequency of segment splitting and fusion operations.

This system design can be regarded as advanced memory management for the PyTorch extension for Ascend. It addresses existing issues of memory segment fragmentation and leverages the latest API updates to redesign mechanisms for memory allocation, garbage collection, and block status updates. We list our contributions as follows:

- We provide a comprehensive analysis to prove that memory fragmentation is a remarkable problem for today's state-of-the-art AI chips.
- We design and implement a function library based on fundamental virtual memory management low-level primitives. Such a library can provide an efficient virtual memory management system for AI chips.
- We design an efficient virtual memory management system to minimize fragmentation for the AI chips, containing complete allocation, free, updating, and garbage collection operations.
- We applied our proposal virtual memory management system for training based on current popular models, and also compared the end-to-end efficiency and performance with Ascend native solutions.
- We also provide a novel algorithm to replace the original best-fit memory allocation algorithm, reducing segment splitting and fusion operations.

II. SYSTEM DESIGN

System Architecture. Memory reuse based solely on expandable segments is inadequate, as it focuses only on individual blocks or segments to determine if there is sufficient space to merge an allocatable block. To address this limitation, we propose monitoring all blocks within the pools. By stitching their subsections together, we can enhance the reuse rate.

Zhaorui Zhang is the corresponding author of this work. Email: zhaorui.zhang@polyu.edu.hk

The new architecture builds upon the concept of expandable segments by introducing an additional component, the fusion pool, which tracks the fusion status of each block. Consequently, the allocation and release of a block will not rely solely on `aclrtMalloc`. The details of the Virtual Memory Management (VMM) process are as follows: ① **aclrtReserveMemAddress**, reserve the virtual memory address. This function will not truly malloc any physical memory from NPU HBM. It just announces that the NPU will keep an address sequence. However, the extraordinary thing is that this step actually takes quite a long time. This is a significant point that the system should optimize. ② **aclrtMallocPhysical** returns a handle that contains information about physical chunks. Within this system design, the granularity of every physical chunk is only 2 MB. Other block sizes will be configured by virtual memory. ③ **aclrtMapMem** maps specific physical handles to the same size as the virtual address. The advantage is that the virtual address pointer can be different from the physical handle. This is the core mechanism of our work. ④ **aclrtUnmapMem** unmaps the relationship between the handle and virtual address. ⑤ **aclrtFreePhysical** release the physical memory space in NPU HBM. ⑥ **aclrtReleaseMemAddress** releases the virtual memory reserved.

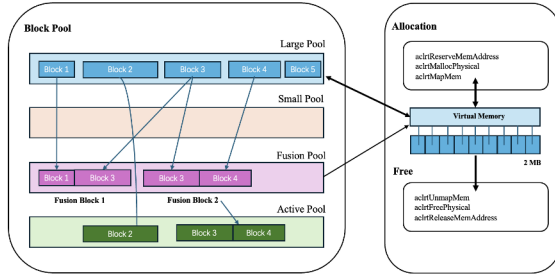


Fig. 1: Architecture Design

Every block initialized via VMM APIs is added to the large pool, which stores both non-integrated and free blocks. When a block merges with other blocks, it is placed into the fusion pool. As illustrated in Fig. 1, fusion blocks can share the same block at different time intervals; for instance, Fusion Block 1 and Fusion Block 2 can both utilize Block 3 when the other is not using it. This occurs when Fusion Block 1 becomes inactive, prompting the large pool to record the free states of Block 1 and Block 3. Concurrently, another allocation may require merging Block 3 and Block 4 to form Fusion Block 2. The fusion pool retains both Fusion Block 1 and Fusion Block 2. In such cases, if Fusion Block 1 detects that its components are being used by Fusion Block 2, it will split the utilized section and remove itself from the fusion pool, with Block 1 being returned to the large pool. This approach maximizes memory reuse across different segments and blocks, in contrast to expandable segments, which can only identify free spaces within a block. The fusion pool can directly release a fusion block once it verifies the free status of each block component and the fusion block is in a complete state. Typically, the fusion block is transferred to the large pool. When all subsections of the virtual address are free, the block is fully released.

Round Size Optimization. Upon examining the allocation logic of the PyTorch extension for Ascend, we observed that Ascend does not implement round-size operations. However, such operations are crucial for ensuring memory alignment. When the allocation size is an odd number, such as 153 MB, the absence of round-size operations results in an allocation block of 256 MB, which is a direct exponential multiple of two. Consequently, the subsequent splitting operation yields a remaining block of 103 MB, which becomes inaccessible to other allocation requests. Additionally, odd-sized memory blocks contribute to the creation of numerous discontinuous fragments, typically around 1 MB. These small fragments are either stored in a small pool or become fragmented blocks within a larger pool. To address this issue, we propose a solution that involves rounding the allocation block size to mitigate misalignment. For instance, if the allocation size is 153 MB, falling between 125 MB and 256 MB, the *Distance* can be calculated by dividing the interval between the lower and upper bounds as $Distance = \lceil \log_2(2^{\lceil \log_2 x \rceil} - 2^{\lfloor \log_2 x \rfloor}) \rceil$, and $A_i = 2^{\lfloor \log_2 x \rfloor} + n \cdot Distance$. In our example, let x represent the allocation size, and *Distance* is set to 16. A_i denotes the i^{th} allocation request from the application. n serves as the coefficient for the number of *Distance* units, ensuring that A_i can be significantly larger than the allocation size. In practice, the upper-layer application can utilize this reserved section from its perspective. Compared to the original strategy, this approach allows more internal memory to be accessed by other requests, thereby enhancing the utilization of NPU memory.

Caching Allocation Optimization. The native caching allocator employs a straightforward best-fit algorithm, directly allocating the required size using `aclrtMalloc`. To further improve the performance, we propose to use the Depth First Search (DFS) approach. The optimal scenario occurs when the `currSumcurrSum` is equal to the target. If the pool does not contain a subset whose sum equals the target, the algorithm will terminate immediately. The condition to maintain is that the `currSumcurrSum` plus a certain element from the pool remains less than the target T . The recursive step involves traversing the depth-first tree of the pool and evaluating the sum of each subset to determine if a subset meeting the target exists. Moreover, if multiple subsets satisfy the target size, selecting the subset with the fewest elements is preferable, as it minimizes stitching operations, virtual memory block reshaping, and remapping.

Release Blocks. Regarding the VMM APIs, the Ascend virtual memory function operates differently from CUDA VMM APIs. When multiple virtual addresses are mapped to physical handles, Ascend must exercise caution when freeing these physical handles. It is essential to ensure that all mapping relationships have been released before freeing a handle. Directly freeing a handle without this precaution can result in errors during the release of virtual addresses. To address this issue, we introduce a reference count for each physical handle. Each mapping operation increments the reference count by one. Conversely, when all blocks within a segment are free and the reference count for each block is zero, the segment can complete the release process.

III. EVALUATION

Our experiment is based on the ModelLink and MindSpeed accelerator libraries and architecture. As indicated in Tab. I, we recommend constructing the testing toolkit on a well-established large DNN training platform, with Python version 3.8 being preferable.

A. Evaluation Methodology

ModelLink integrates features from Megatron-LM, including pipeline parallelism, tensor parallelism, and distributed training systems. However, the efficiency of our system is best demonstrated through the fine-tuning of pre-trained models. To maintain the complexity of the task, we have opted for full parameter fine-tuning and Low-Rank Adaptation (LoRA).

TABLE I: Experimental Environment Details

Item	Detail
Device	Ascend 910B3 (64GB) * 4
CANN Version	8.0.0.RC1
PyTorch Extension	2.1.0.RC1
ModelLink Version	1.0.RC1
Megatron-LM Version	core_r0.6.0
MindSpeed Version	1.0.RC1_core_r0.6.0
Python Version	3.8
Apex Version	0.1.0

We utilize the OPT and LLaMA series models, which exemplify state-of-the-art training scenarios for contemporary models. Ascend provides several dataset examples, from which we derive insights, focusing particularly on Alpaca.

Our baseline is the original PyTorch Extension 2.1.0 RC1, featuring the expandable segment mechanism, recognized as the most stable version currently available. We analyze the reserved and active memory for each training epoch summary. Reserved memory refers to the amount allocated by the system from physical memory, while active memory denotes the actual utilization by the system. Notably, the current active memory value consistently falls below the reserved memory. Consequently, the peak value serves as a critical reference point. The memory utilization ratio is calculated as the active memory divided by the reserved memory as follows.

$$Utilization\ Ratio = \frac{Peak\ Active\ Memory}{Peak\ Reserved\ Memory} \quad (1)$$

On the other hand, the fragmentation ratio can be represented by $Fragmentation\ Ratio = 1 - Utilization\ Ratio$.

B. Memory Performance Results

The end-to-end performance metrics were obtained using PyTorch's profiling memory summary tools. We evaluated two popular models, LLaMA2 [23] and OPT [24], shown in Fig. 2. The results for the LLaMA-2-7B (Fig. 2(a)), demonstrate that our strategy outperforms the expandable segment approach due to more efficient memory fusion. Ascend supports LoRA, and the performance trend is similar to that of full-parameter fine-tuning. However, thanks to efficient memory optimization, the Ascend native PyTorch scheduler exhibits significantly greater efficiency compared to the CUDA platform. Specifically, for the LLaMA2-7B model on CUDA, the native

scheduler achieves approximately 73% memory utilization, whereas Ascend achieves 90%. We investigated the reasons behind this remarkable efficiency, which appear to be related to the hardware architecture. With the Expandable Segment approach, utilization can reach 94%, an extraordinary achievement. Our solution further improves this to 96%, addressing the limitations of the Expandable Segment by maximizing memory reuse across different segments and blocks.

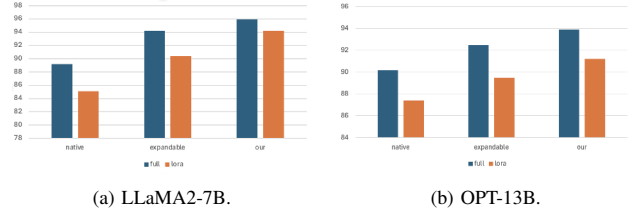


Fig. 2: The Memory Utilization of Different Models.

Changing models can significantly impact results, as diverse models have varying segment allocation size requirements, leading to different fragmented memory sizes within the large pool. Complex allocation sizes can reduce the performance of native allocators; however, this effect is minimal in our solution. The utilization ratio between models such as LLaMA and OPT varies considerably due to differing mixed precision training requirements. The memory saving ratio is approximately 2% per device, equating to about 0.64 GB to 1.2 GB.

Additionally, we examined the fine-tuning process and found that our method consistently outperforms the Expandable Segment approach, with differences ranging from 1% to 2% in each iteration. Fig. 3 illustrates the iterations of the LLaMA2-7B model. As iterations progress to a certain level, memory utilization stabilizes. However, active and reserved memory continue to fluctuate, as recorded by memory profiling. The difference between active and reserved memory is dynamic, varying in size. Depending on training requirements, reserved memory can be retained for inference, dynamically enhancing available memory for different users. Conversely, if training task throughput is prioritized, maintaining reserved memory may lead to inefficiencies and wasted resources. During the initial iterations of a training task, memory utilization is relatively low. This is because the upper-level application attempts to reserve sufficient memory for future use. At the beginning stage, active memory occupies only a portion of the reserved memory. As iterations progress, larger allocations are introduced into the scheduler, leading to peak utilization ratios.

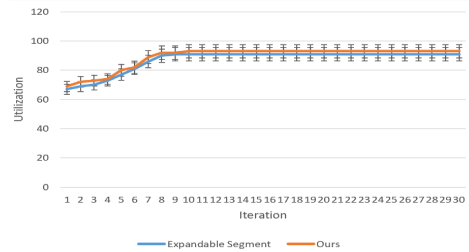


Fig. 3: Memory Utilization for Different Iterations.

We have optimized the rounding size and the greedy algorithm within the Best-fit strategy. By examining allocation details and recording the states of both small and large pools, we have achieved significant improvements. The optimization of the rounding size effectively reduces the volume of the small pool. For instance, with the LLaMA2-7B model, the peak reserved size of the small pool originally measured 112 MB, but was reduced to 74 MB with the rounding size optimization. Although the improvement may seem modest, it is important to note that this is an end-to-end test, and the optimization of the greedy algorithm can increase the space of the small pool. Consequently, the rounding size method demonstrates much greater efficiency than previously reported experimental data. We propose that this method strikes a balance between memory alignment and availability. If the rounding size is based solely on the request size, it may lead to the creation of new allocation requests. Conversely, adhering to the original strategy can result in significant internal fragmentation and memory misalignment. Thus, this strategy has the potential to enhance memory utilization efficiency and reduce the frequency of memory release operations.

TABLE II: Split and Stitch Operation Times for Different Strategies

Operation	Split	Stitch
Greedy Algorithm	895	1074
Optimize Strategy	833	942

As demonstrated in Tab. II, the number of stitch operations exceeds that of split operations across various algorithms, primarily due to the prevalence of stitch operations. By employing an optimization strategy, both split and stitch operations are reduced. Identifying the appropriate subset within a large pool can also decrease the number of blocks. During the development of the actual algorithm, the optimization strategy not only ensures that the sum of the subset equals the target but also verifies that the difference between the fusion block and the target size is less than 20 MB. If this difference is less than 1 MB, the remaining block is stored in a small pool. For remaining sizes from split operations between 1 MB and 20 MB, these sizes are often utilized for stitching and remain active, aligning with the management system's objectives. Furthermore, the final results are consistent with our initial hypothesis.

IV. CONCLUSION

In this study, we developed a memory management optimization strategy tailored for the Ascend. The primary concept was inspired by the PyTorch extension mechanism, the expandable segment approach, and advanced work on the CUDA platform, specifically GMLake. Our approach leverages VMM APIs to maximize memory reuse by incorporating a fusion block pool and addressing various allocation and release scenarios. We identified existing drawbacks and proposed corresponding optimization solutions. Experimental results indicate that our memory management system can save between 1% and 3% of memory. In future work, we aim to further refine this strategy to enhance its performance and applicability for both training and inference tasks.

V. ACKNOWLEDGMENT

This work was supported in part by the National Natural Science Foundation of China under Grant 62302420 and in part by the Hong Kong Innovation and Technology Support Programme (ITP) under Grant ITP/012/25LP.

REFERENCES

- [1] Zhang Z, Ji Z, Wang C. Momentum-driven adaptive synchronization model for distributed DNN training on HPC clusters[J]. *Journal of Parallel and Distributed Computing*, 2022, 159: 65-84.
- [2] Zhang Z, Wang C. SaPus: Self-adaptive parameter update strategy for DNN training on Multi-GPU clusters[J]. *IEEE Transactions on Parallel and Distributed Systems*, 2021, 33(7): 1569-1580.
- [3] Zhang Z, Wang C. MIPD: An adaptive gradient sparsification framework for distributed DNNs training[J]. *IEEE Transactions on Parallel and Distributed Systems*, 2022, 33(11): 3053-3066.
- [4] Xu H, Zhang Z, Di S, et al. FedFa: A Fully Asynchronous Training Paradigm for Federated Learning[C], 33rd International Joint Conference on Artificial Intelligence, IJCAI 2024: 5281-5288.
- [5] Zhang Z, Di S, Zhao K, et al. FedCSpc: A Cross-Silo Federated Learning System with Error-Bounded Lossy Parameter Compression[J]. *IEEE Transactions on Parallel and Distributed Systems*, 2025.
- [6] Zhang Z, Di S, Liu B, et al. FedEFsz: Fair Cross-Silo Federated Learning System with Error-Bounded Lossy Compression[J]. *IEEE Transactions on Parallel and Distributed Systems*, 2025.
- [7] Ma S, Zhang Z, Di S, et al. MoE-Compression: How the Compression Error of Experts Affects the Inference Accuracy of MoE Model?[J]. *arXiv preprint arXiv:2509.07727*, 2025.
- [8] Shen, Li, et al. "On Efficient Training of Large-Scale Deep Learning Models." *ACM Computing Surveys* 57.3 (2024): 1-36.
- [9] Rajbhandari, Samyam, et al. "Zero: Memory optimizations toward training trillion parameter models." *International Conference for High Performance Computing, Networking, Storage and Analysis*. 2020.
- [10] Shoenyi, Mohammad, et al. "Megatron-lm: Training multi-billion parameter language models using model parallelism." *arXiv preprint arXiv:1909.08053* (2019).
- [11] Huawei, CANN 8.0.RC3 AscendCL Application Software Development Guide (C and C++, Inference), Huawei Technologies Co., Ltd, 2024
- [12] Liao, Heng, et al. "Ascend: a scalable and unified architecture for ubiquitous deep neural network computing: Industry track paper." 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA). IEEE, 2021.
- [13] Zhang, Susan, et al. "Opt: Open pre-trained transformer language models, 2022." URL <https://arxiv.org/abs/2205.01068> 3 (2023): 19-0.
- [14] Kim, Sehoon, et al. "Full stack optimization of transformer inference: a survey." *arXiv preprint arXiv:2302.14017* (2023).
- [15] Han, Husheng, et al. "TensorTEE: Unifying Heterogeneous TEE Granularity for Efficient Secure Collaborative Tensor Computing." 2024.
- [16] Weingram, Adam, et al. "xCCL: A Survey of Industry-Led Collective Communication Libraries for Deep Learning." *Journal of Computer Science and Technology* 38.1 (2023): 166-195.
- [17] Huawei Technologies Co., Ltd, Parameter Replica Reuse, 2024, <https://gitee.com/ascend/MindSpeed/blob/master/docs/features/reuse-fp32-param.md>
- [18] Huawei Technologies Co., Ltd, Megatron Distributed Optimizer, 2024, <https://gitee.com/ascend/MindSpeed/blob/master/docs/features/distributed-optimizer.md>
- [19] Kusumoto, Mitsuru, et al. "A graph theoretic framework of recomputation algorithms for memory-efficient backpropagation." *Advances in Neural Information Processing Systems* 32 (2019).
- [20] HW Co., Ltd, Swap Attention, 2024, https://gitee.com/ascend/MindSpeed/blob/master/docs/features/swap_attention.md
- [21] Guo, Cong, et al. "GMLake: Efficient and Transparent GPU Memory Defragmentation for Large-scale DNN Training with Virtual Memory Stitching." *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2. 2024.
- [22] Hu, Edward J., et al. "Lora: Low-rank adaptation of large language models." *arXiv preprint arXiv:2106.09685* (2021).
- [23] Li, Chen, et al. "Common 7b language models already possess strong math capabilities, 2024." URL <https://arxiv.org/abs/2403.04706>.
- [24] Zhang, Susan, et al. "Opt: Open pre-trained transformer language models, 2022." URL <https://arxiv.org/abs/2205.01068> 3 (2023): 19-0.